

* 8051 Instruction Set:-* Data Transfer Instructions:-(i) MOVSyntax:- MOV <dest-byte>, <src-byte>

- This instruction moves or copies the contents of the source byte to destination byte.

- The source byte is not affected.

* - By this instn no other register or flag is affected.

e.g. ~~MOV R0, #30H~~

MOV R0, #30H

Here 30H value / address is moved in R0-register.

(a) MOV A, Rn Byte=1, Cycle=1

- move content of Rn (where n=0 to 7) -regi. into Accumulator.

(b) MOV A, direct Byte=2, Cycle=1

- moves the direct data to accumulator.

* (c) MOV A, ACC is not valid Instruction.(d) MOV A, @Ri Byte=1, Cycle=1

- move content of memory location pointed by Ri-register to accumulator.

(e) MOV A, #data Byte=2, Cycle=1

- moves immediate data to A.

(f) MOV Rn, A Byte=1, Cycle=1

- moves accumulator contents to Rn -regi. where n=0 to 7.

(g) MOV Rn, direct Byte=2, Cycle=2

- moves direct data to Rn -regi. where n=0 to 7.

⑩

⑥ MOV Rn, #data Byte = 2, cycle = 1

- moves immediate data in Rn reg. where $n = 0$ to 7.

⑦ MOV direct, A Byte = 2, cycle = 1

- moves 'A' contents into direct memory locⁿ.

⑧ MOV direct, Rn Byte = 2, cycle = 2

- move content of Rn reg. ($n = 0$ to 7) to direct location.

⑨ MOV direct, direct Byte = 3, cycle = 2

- move direct data to ~~data~~ direct memory location or data.

⑬ MOV direct, @Ri Byte = 2, cycle = 2

- moves the contents of memory locⁿ pointed by Ri reg. (where $i = 0$ or 1) to direct locⁿ.

⑭ MOV direct, #data Byte = 3, cycle = 2

- move immediate data to direct locⁿ.

⑮ MOV @Ri, A Byte = 1, cycle = 1

- move accumulator content to memory locⁿ pointed by Ri reg. where ($i = 0$ or 1).

⑯ MOV @Ri, direct Byte = 2, cycle = 2

- moves direct data to memory locⁿ pointed by Ri reg. where $i = 0$ or 1,

⑰ MOV @Ri, #data Byte = 2, cycle = 1

- moves immediate data to memory locⁿ pointed by Ri reg. where $i = 0$ or 1.

* (1.1) MOV <dest-bit>, <src-bit>

(11)

- This instruction moves source bit to destination bit.

- One of the operands must be carry flag, and another may be any directly addressable bit.

- NO other register or flag is affected.

e.g. MOV P1.1, C

- Here content of carry flag is copied to P1.1 bit.

(a) MOV C, bit Byte = 2, cycle = 1

- move content of bit to carry flag.

(b) MOV bit, C Byte = 2, cycle = 1

- move content of carry flag in bit address.

* (1.2) MOV DPTR, #data16

- move 16-bit immediate data to data pointer regi.

- 16 bit data/constant is loaded into second byte i.e. DPH (High-order byte), while third byte i.e. DPL (Low-order byte).

- NO flags are affected.

* - This is the only instruction which moves 16-bits data at once.

e.g. MOV DPTR, #1223H

- Immediate data 1223H is moves into DPTR

- Byte = 3 , cycle = 2

* (2) MOVC A, @A+<base-reg>

(2)

- This instⁿ load accumulator with code byte or constant from program memory.

- The address of the byte fetched is the sum of the original unsigned 8-bit accumulator content and the content of 16-bit base register, which may be either data pointer or program counter.

⑫

- No flags are affected.
 e.g. `MOVC A, @A+PC`

① `MOVC A, @A+DPTR` Byte=1, cycle=2

- moves code memory i.e. content of the sum of accumulator and DPTR to accumulator.

② `MOVC A, @A+PC` Byte=1, cycle=2

- move the content of code memory i.e. the sum of Accumulator & PC to accumulator.

- The PC is incremented to the address of ^{above} ~~following~~ instⁿ before being added to A, otherwise base regi. is not altered.

- Sixteen bit addition is performed so a carry-out from low order 8-bits may propagate through higher order bits.

* ③ `MOVX <dest-byte>, <src-byte>`

- This instⁿ moves external source byte to internal memory i.e. accumulator or registers.

- First type - The contents of R0 or R1 in current regi. bank provides an 8-bit address multiplex with data on P0. Eight bits are sufficient for external I/O expansion decoding or for small RAM array. For larger arrays, any o/p port pins can be used to o/p higher-order address bits. These pins are controlled by an o/p instⁿ preceding `MOVX`.

- Second type - Data pointer generates a 16-bit address. P2 o/p higher order eight address bits i.e. DPH content, while P0 gives DPL content.
 e.g. ~~MOVX~~ `MOVX A, @R1`

④ `MOVX A, @Ri` Byte=1, Cycle=2

- moves content of memory locⁿ of external RAM pointed by Ri-regi. where i=0 or 1 to A-regi.

⑥ MOVX A, @DPTR

Byte = 1 Cycle = 2

- Moves contents of external memory locⁿ pointed by DPTR to accumulator.

⑦ MOVX @Ri, A

Byte = 1, cycle = 2

- moves contents of accumulator to external memory locⁿ pointed by Ri-regi. where i = 0 or 1.

⑧ MOVX @DPTR, A

Byte = 1, cycle = 2

- moves contents of accumulator to external memory locⁿ pointed by DPTR.

● ④ ~~POP~~ PUSH direct

Byte 2, cycle = 2

- This instⁿ push the content into stack memory.
- The stack pointer is incremented by one.
- The contents of indicated variable is then copied into internal RAM locⁿ pointed by stack pointer.
- No flags are affected.

e.g. PUSH DPL

- Here the contents of DPL-regi. is copied to memory pointed by SP.

● ⑤ POP direct

Byte = 2, Cycle = 2

- This instⁿ POP out content from stack memory
- The contents of internal RAM locⁿ addressed by SP is read & SP is decremented by one.
- The value read is then transferred to directly addressed byte.
- NO flags are affected.

e.g. POP DPL

- Here the contents of memory locⁿ pointed by SP is moved to the DPL-regi.

⑥ XCH A, <byte>

(14)

- This instruction exchanges the contents of accumulator with indicated variable.

- The source / destination operand can be register, direct or register-indirect addressing.

e.g. XCH A, @R0

- It exchanges the content of accumulator with memory locⁿ pointed by R0 register.

② XCH A, R_n

Byte = 1, Cycle = 1

- Exchanges content of accumulator with contents of R_n register, where n = 0 to 7.

③ XCH A, direct

Byte = 2, Cycle = 1

- Exchanges content of accumulator with direct data / locⁿ.

④ XCH A, @R_i

Byte = 1, Cycle = 1

- Exchanges contents of accumulator with contents of memory locⁿ pointed by R_i register, where i = 0 or 1.

⑦ XCHD A, @R_i

Byte = 1, Cycle = 1

- This instruction exchanges low-order nibble of Accumulator, generally representing hexadecimal or BCD digit, with that of internal RAM locⁿ pointed by R_i register.

* - The high-order nibbles (i.e. bits 7-4) of each register are not affected.

- No flags are affected.

e.g. XCHD A, @R0

- This instⁿ exchanges low nibble (i.e. bit 0-3) of accumulator with content of memory locⁿ pointed by R0 register.

* Arithmetic Instruction Set:

① ADD A, <Src-byte>

- This instruction adds source byte with contents in accumulator.
- The result is stored in Accumulator.
- The carry flag and Auxiliary carry flag are set, ~~and~~ if there is carry-out from bit 7 or bit 3 respectively, ~~and~~ otherwise it is cleared.
- When adding unsigned integers, the carry flag indicates an overflow occurred.
- OV is set if there is a carry-out of bit 6 but not out of bit 7 or carry out of bit 7 but not bit 6, otherwise it is cleared.

② ADD A, Rn

Byte = 1, Cycle = 1

- Adds content of Rn where n = 0 to 7 with accumulator & stores result into Accumulator.

③ ADD A, direct

Byte = 2, Cycle = 1

- Adds contents of direct locⁿ or data with accumulator & stores result into Accumulator.

④ ADD A, @Ri

Byte = 1, Cycle = 1

- Adds contents of memory locⁿ pointed by Ri - regi where i = 0 or 1 with A & stores result into A.

⑤ ADD A, #data

Byte = 2, Cycle = 1

- Adds immediate data with A & stores result into A.

⇒ When adding signed integers, OV indicates -ve number produced as sum of two +ve operands or a +ve sum from two -ve operands.

* ⇒ Four source operand addressing modes are allowed: Register, direct, register indirect & immediate.

⑩ ② ADDC A, <src-byte> -

- This instⁿ simultaneously adds the byte variable indicated, the carry flag & accumulator contents.

- The result is stored in Accumulator.

* - The carry flag is set, if there is carry out from bit 7, otherwise cleared.

* - The auxiliary carry flag is set, if there is carry out from bit 3, otherwise cleared.

- When adding unsigned integers, the carry flag indicates an overflow occurred.

- OV is set if there is carry out of bit 6 but not out of bit 7, or carry out of bit 7 but not out of bit 6, otherwise OV is cleared.

- When adding signed integers, OV indicates a negative no. produced as sum of two +ve nos. or a positive sum from two -ve operands.

* - Four source operand addressing modes are allowed - register, direct, register-indirect or immediate.

e.g. ADDC A, R0

- This instⁿ adds contents of R0 & carry flag with accumulator & stores result in A, and both carry flag & OV flag is set according to data.

③ ADDC A, Rn Byte=1, Cycle=1

- Adds content of Rn regi. where n=0 to 7 & carry flag with A. and stores result in A.

④ ADDC A, direct Byte=2, Cycle=1

- Adds content of direct data/locⁿ & carry flag with A. and stores result in A.

⑤ ADDC A, @Ri Byte=1, Cycle=1

- Adds contents of memory locⁿ pointed by Ri-regi. where i=0 or 1 & carry flag with accumulator & stores result in accumulator.

(17) (d) ADDC A, #data Byte=2, cycle=1

- adds immediate data & carry flag with accumulator & stores result in Accumulator.

(3) (e) SUBB A, <src-byte>

- Subtracts source operand and carry flag together from Accumulator & stores result into Accumulator.

- SUBB sets carry (Borrow) flag if a borrow is needed for bit 7 and otherwise carry flag is cleared.

- Auxiliary Carry (AC) is set if a borrow is needed for bit 3 and otherwise AC is cleared.

- OV is set if a borrow is needed into bit 6 but not in bit 7 or into bit 7 but not in bit 6.

- when subtracting signed integers, OV indicates a -ve no. produced when a -ve value is subtracted from +ve value or +ve result when a +ve no. subtracted from -ve no.

* - The source operand allows four addressing modes:- Register, Direct, Register-Indirect, and immediate.

E.g. SUBB A, R2

- Subtracts contents of R2 register & C flag from A & stores result into A.

(a) SUBB A, Rn Byte=1, Cycle=1

- Subtracts Rn register where n=0 to 7 contents and C flag from A & stores result into A.

(b) SUBB A, direct Byte=2, Cycle=1

- Subtract direct data & C flag from A & stores result into A.

(c) SUBB A, @Ri Byte=1, Cycle=1

- Subtracts contents of Ri regi. where i=0 or 1 and C flag from A & stores result into A.

(18)

(d) SUBB A, #data Byte=2, cycle=1

- Subtracts immediate data & C Flag from Accumulator & Stores result in A.

(4) INC <byte>

- Increments source byte by 1.

- An original value of 0FFH overflows to 00H.

- NO flags affected.

* - Three addressing modes are allowed:-
Register, Direct, Register-Indirect.

- When this instⁿ is used to modify an I/O port, value used as the original port data will be read from I/O data latch, not I/O pins.

e.g. INC R0

- Increment value of R0 content by 1.

(a) INC A

Byte=1, cycle=1

- Increment content of A by 1.

(b) INC Rn

Byte=1, cycle=1

- Increment content of Rn regi. where n=0 to 7 by 1.

(c) INC direct

Byte=2, cycle=1

- Increment direct data by 1.

(d) INC @Ri

Byte=1, cycle=1

- Increment content of memory locⁿ pointed by Ri regi. where i=0 or 1 by 1.

* (4.1) INC DPTR

Byte=1, cycle=2

- Increments the 16-bit data pointer by 1.

- A 16 bit increment (modulo 2^{16}) is performed, and an overflow of low-order byte of DPTR i.e. DPL from 0FFH to 00H increments the high-order byte i.e. DPH.

- No Flags are affected.

⑤ DEC byte

- Decrements source byte by 1.
- An original value of 00H underflows to 0FFH.
- No Flags are affected.
- Four operand addressing modes are allowed - accumulator, register, direct, register-indirect.
- e.g. DEC R0
- Decrement contents of R0 by 1.

① DEC A Byte=1, cycle=1

- Decrements contents of A by 1.

② DEC Rn Byte=1, cycle=1

- Decrements contents Rn-regi. where n=0 to 7 by 1.

③ DEC direct Byte=2, cycle=1

- Decrements direct data by 1.

④ DEC @Ri Byte=1, cycle=1

- Decrements content of memory locⁿ pointed by Ri where i=0 or 1 by 1.

⑥ MUL AB Byte=1, cycle=4

- Multiplies unsigned 8-bit integers in the accumulator or and register B.
- The lower byte of 16-bit product is left in the accumulator, and higher byte in B-regi.
- If product is greater than 255 (0FFH), then overflow flag is set, otherwise it is cleared.
- The carry flag is always cleared.

⑦ DIV AB Byte=1, cycle=4

- Divides unsigned 8-bit integer in Accumulator by unsigned 8-bit integer in regi. B.

(20)

* - The accumulator receives integer part of quotient & regi. B receives integer Remainder.

- The carry & OV flags are cleared.

* - Exception - IF B contents 00H, The values returned in A & B are undefined and OV Flag is set.

(8)

DA A :- Decimal - Adjust Accumulator for Addition.

- Adjusts the 8-bit value in Accumulator resulting from earlier addition of two variable (each in packed-bcd format), producing two four-bit digits.

- IF Accumulator bits 3 through 0 are greater than 9 or if AC Flag is one, six is added to accumulator producing proper BCD digit in lower nibble. This internal addition sets the carry flag if a carry-out of lower nibble propagates to higher nibble, but it does not clear carry flag.

- IF carry flag is now set or if Higher Nibble bits now exceed nine, those higher nibble bits are incremented by six, producing proper BCD in higher nibble. This again set carry flag & carry flag is not cleared.

- Thus carry flag indicates if sum of the original two BCD variables is greater than 100, allowing multiple precision decimal addition.

- OV is not affected.

- This instⁿ performs the decimal conversion by adding 00H, 06H, 60H or 66H to Accumulator, depending on initial Accumulator & PSW conditions.

* - DA A cannot convert hexadecimal no. in Accumulator to BCD, nor does DA A apply to decimal subtraction.

- Byte = 1 , Cycle = 1

* (3) Logical Instruction Set :-

① ANL <dest-byte>, <src-byte>

- It performs bitwise logical AND operation between destination byte and source byte.
- The result is stored in destination operand.
- NO flags are affected.
- Two operands allows six addressing mode combinations.
- When destination is Accumulator, the source can use register, direct, register-indirect or immediate mode.
- When destination is direct address then source can be Accumulator or immediate data.

e.g. content of R0 = 01101111B

ANL A, R0

- It logically AND content of accumulator with R0 $\rightarrow$ reg.

② ANL A, Rn

Byte = 1, Cycle = 1

- Bitwise logically AND contents of A & Rn $\rightarrow$ reg. where $n = 0$ to 7 & stores result into A.

③ ANL A, direct

Byte = 2, Cycle = 1

- Bitwise AND content of A & direct data and stores result in A.

④ ANL A, @Ri

Byte = 1, Cycle = 1

- Bitwise AND contents of A and memory locⁿ pointed by Ri $\rightarrow$ reg. where $i = 0$ or 1, & result stored in A.

⑤ ANL A, #data

Byte = 2, Cycle = 1

- Bitwise AND contents of A and immediate data & stores result in A.

⑥ ANL direct, A

Byte = 2, Cycle = 1

- Bitwise AND contents of direct address & A and stores result to direct address.

22

(F) ANL direct, #data Byte=3, Cycle=2

- Bitwise logically ~~and~~ AND content of direct address & immediate data & stores result in direct address.

(1.1) ANL C, <src-bit>

- If the Boolean value of source bit is logical '0', then ANL C clears carry flag; otherwise, this instⁿ leaves carry flag in its current state.

- A slash (/) preceding the operand in the assembly language indicates that the logical complement of addressed bit is used as the source value, but source bit itself is not affected.

- No other flags are affected.

- only direct addressing mode is allowed for the source operand.

E.g. ANL C, ACC.7

- Bitwise AND carry flag with accumulator bit 7.

(a) ANL C, bit Byte=2, Cycle=2

- AND Carry Flag with specified bit address & result ~~is~~ stored in carry flag.

(b) ANL C, /bit Byte=2, Cycle=2

- AND carry Flag with specified bit/bit address ~~is~~ complement & stores result in carry flag.

(2) ORL <dest-byte>, <src-byte>

- It perform bitwise logical OR operation between destination byte and source byte.

- The result is stored ~~in~~ destination byte.

- No flags are affected.

- Two operands allow six addressing modes combinations.



- when destination is Accumulator, the source can use register, direct, register-indirect or immediate mode.

- when destination is a direct address, the source can be accumulator or immediate data.

e.g. ORL A, R0

① ORL A, R_n Byte=1, Cycle=1

- Bitwise logically OR contents of A & R_n regi. where n=0 to 7, & stores result into A.

② ORL A, direct Byte=2, Cycle=1

- Bitwise logically OR contents of A & direct address/data and stores result in A.

③ ORL A, @R_i Byte=1, Cycle=1

- Bitwise logically OR contents of A & memory location pointed by R_i regi. where i=0 or 1 and stores result into A.

④ ORL A, #data Byte=2, Cycle=1

- Bitwise logically OR contents of A & immediate data & stores result into A.

⑤ ORL direct, A Byte=2, Cycle=1

- Bitwise logically OR contents of direct address/data & A and stores result in direct address.

⑥ ORL direct, #data Byte=3, Cycle=2

- Bitwise logically OR contents of direct address and immediate data and stores result in direct address

i.e. direct $\leftarrow$ direct $\vee$ #data.

24 (2.1) ORL C, <src-bit>

- Set carry flag if the Boolean value is a logical 1; leave the carry in it's current state otherwise.
- The slash (/) preceding the operand in assembly language indicates that logical complement of addressed bit is used as the source value, but source bit itself is not affected.
- No ~~the~~ other flags are affected.
- e.g. ORL C, ACC.5
- Bitwise logically OR carry flag with accumulator bit 5.

(a) ORL C, bit Byte = 2, cycle = 2

- Bitwise logically OR carry flag with source bit & stores result in carry flag.

(b) ORL C, /bit Byte = 2, cycle = 2

- Logically OR carry flag with complement of bit address & stores result in carry flag.

(3) XRL <dest-byte>, <src-byte>

- It performs bitwise logical Exclusive-OR operation between destination byte & source byte & stores result in destination byte.

- No flag affected.

- Two operands allows six addressing mode combinations, when destination is accumulator, the source can be ~~accumulator or~~ register, direct, register-indirect or immediate.

- When destination is direct address, the source can be accumulator or immediate data.

e.g. XRL A, Rn

- Bitwise logical Ex-OR content of A with Rn regi. where n = 0 to 7 & stores result in A.

① XRL A, R_n Byte = 1, cycle = 1
 - Bitwise logically EX-OR content of A & R_n
 where $n = 0$ to 7 & stores result in A.

② XRL A, direct Byte = 2, cycle = 1

③ XRL A, @R_i Byte = 1, cycle = 1

④ XRL A, #data Byte = 2, cycle = 1

⑤ XRL direct, A Byte = 2, cycle = 1

- Bitwise logically EX-OR content of direct address/data with A & store result in direct address.

⑥ XRL direct, #data Byte = 3, cycle = 2

- Bitwise logically EX-OR content of direct address/data with immediate data & stores result in direct address.

④ CLR A Byte = 1, cycle = 1

- This instⁿ clears the content of accumulator i.e. sets all bit to 0.

- No flags are affected.

④ CLR bit - This is bit manipulation instⁿ.

- This instⁿ clears indicated bit i.e. reset to 0.

- No other flags are affected.

- CLR can operate on the carry flag or any directly addressable bit.

e.g. CLR P1.1

- It clears pin P1.1 i.e. $P1.1 = 0$.

① CLR C Byte = 1, cycle = 1

- Clears carry flag.

② CLR Bit Byte = 2, cycle = 1

- Clears indicated bit/bit address.

26 (5) CPL A

Byte = 1, Cycle = 1

- This instⁿ logically complements each bit of accumulator (only one's complement). Bit which previously contains a '1' are changed to '0' & vice-versa.

- NO flags are affected.

operation CPL

$$(A) \leftarrow \overline{(A)}$$

(5i) CPL bit - This is bit manipulation instⁿ

- This instⁿ complements specified bit. A bit that had been '1' changed to '0' and vice-versa.

- NO other flags are affected.

- CLR can operate on carry flag or any directly addressable bit.

(a) CPL C

Byte = 1, Cycle = 1

- Complements content of carry flag.

(b) CPL bit

Byte = 2, Cycle = 1

- complements specified bit address.

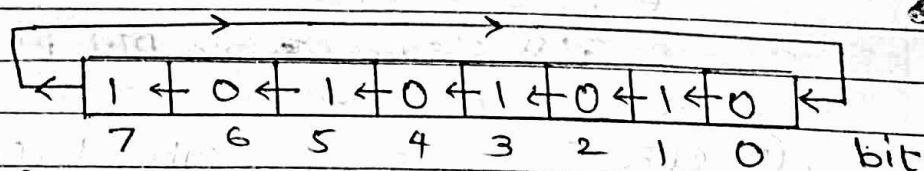
(6) RL A - rotate Accumulator Left

Byte = 1, Cycle = 1

- This instⁿ rotates 8-bits in accumulator to left by one position.

- ~~Bit~~ Bit 7 is rotated into '0' position

- NO flags are affected.



(7) RLC A - Rotate A Left through carry flag

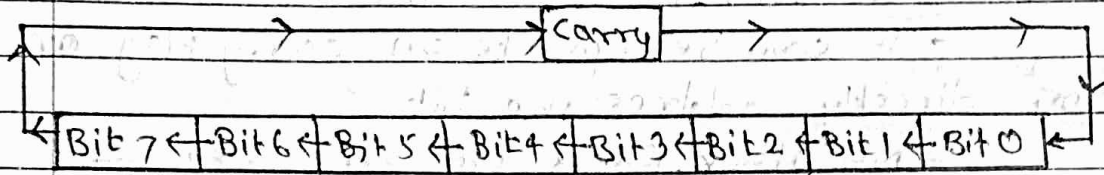
Byte = 1, Cycle = 1

- This instⁿ rotates 8 bits in accumulator through carry flag to left by one position.

- Here Bit 7 is moves into the carry flag and original content of carry flag is ~~is~~

moves into bit 0 position

- No other flags are affected

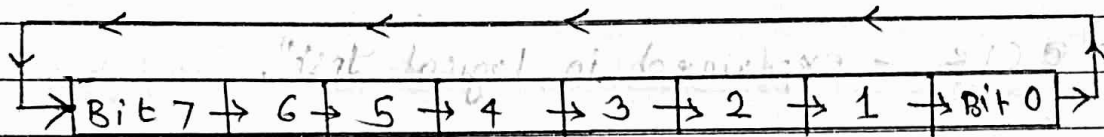


⑧ RR A - Rotate A right Byte=1, Cycle=1

- This instⁿ rotates 8-bits in Accumulator to right by one bit.

- Bit 0 is rotated in Bit 7 position.

- No flags are affected.

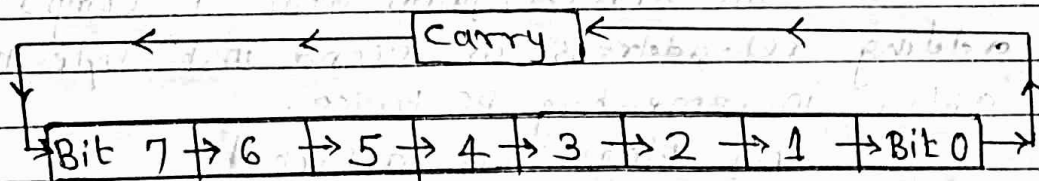


⑨ RRC A - Rotate A Right through carry Byte=1, Cycle=1

- This instⁿ rotates 8-bits in Accumulator to right through carry flag by one bit.

- Bit 0 moves in carry flag & original content in carry flag moves in bit 7 position

- No other flags are affected.



⑩ SWAP A Byte=1, Cycle=1

- This instⁿ swaps or interchanges the lower and higher nibble of the accumulator i.e. swaps bits 3 to 0 with bit 7 to 4.

- The operation can also be thought of as 4-bit rotate instⁿ.

- No flags are affected.

28 * ④ Bit manipulation Instructions -

① SETB <bit>

- It sets indicated bit to One.
- It can be operate on carry flag or any directly addressable bit.
- NO other flags are affected.

① SETB C

Byte=1, Cycle=1

- This instⁿ sets carry flag to 1.

② SETB bit

Byte=2, Cycle=1

- This instⁿ sets bit/bit address to 1.

② CLR - explained in logical instⁿ.

③ MOV - explained in data transfer instⁿ.

④ JC rel-addr

Byte=2, Cycle=2

- Jumps if carry flag is set.
 - If carry flag is set, this instⁿ branches/jump to indicated rel-addr otherwise, it proceeds with next instⁿ.
 - The branch/jump destⁿ is computed by adding rel-address in second instⁿ byte to the PC, after incrementing PC twice.
 - NO flags are affected.
- e.g. SETB C.
JC L1

- Here carry flag is set to 1, so JC branches to address ~~show~~ pointed by L1 label.

⑤ JNC rel-addr

Byte=2, Cycle=2

- Jumps to rel-addr if carry is zero.
- If content of carry flag is Zero ('0'), then JNC branches to rel-addr indicated, otherwise, it proceeds with the next instⁿ.
- The branch destⁿ is computed by adding

rel-address in second instⁿ byte to PC, after incrementing PC twice to point next instⁿ.

- The carry Flag is not modified.

e.g. CLR C

JNC L1

- Here carry flag is set to 0. the JNC branches to memory address pointed by Label L1.

⑥ JB bit, rel-address Byte=3, cycle=2

- If specified bit is one, then JB Jump to rel-address indicated, otherwise, it proceeds with next instⁿ.

- The bit tested is not modified.

- NO Flags are affected.

e.g. JB P1.1, L1

- Here if $P1.1 = 1$ the JB jump to memory location pointed by Label L1.

⑦ JNB bit, rel-address Byte=3, cycle=2

- If specified bit is set to '0', JNB jump to indicated rel-address, otherwise, it proceeds with the next instⁿ.

- The bit tested is not modified.

- NO Flags are affected.

e.g. ~~IF~~ JNB P2.1, L1

- Here if $P2.1 = 0$, then JNB jump to the memory location pointed by Label L1.

⑧ JBC bit, rel-address Byte=3, cycle=2

- Jumps if bit is set & clears bit

- If specified bit in instⁿ is One, then JBC branches to indicated rel-address in instⁿ, otherwise it proceeds with next instⁿ.

* - After branch specified bit is cleared to zero.

- NO Flags are affected.

e.g. JBC P1.2, L1

- Here if $P1.2 = 1$, then JBC jumps

30 clears P1.2 to 0, then jumps to memory locⁿ pointed by Label L1.

⑨ ANL - Already explained in Logical Instⁿ.

⑩ ORL - Already explained in logical instⁿ.

⑪ CPL - Already explained in logical instⁿ.

* ⑤ Branch - Jump and Call Instⁿ :-

① ~~LJMP~~ LJMP addr16 Byte=3, cycle=2

- Long jump cause an unconditional branch to indicated 16-bit address, by loading higher & lower byte of PC (respectively) with ~~second~~ second & third instⁿ byte.

- The destination may therefore be anywhere in the full 64K program memory address space.

- NO Flags are affected.

e.g. LJMP ADDR

- Here LJMP branches to address specified by ADDR Label.

② AJMP addr11 :- Absolute Jump Byte=2, cycle=2

- It transfers program execution to indicated 11-bit destination Address, which is formed at run-time by concatenating the higher five bits of PC (after incrementing PC twice), opcode bits 7 through 5 & the second byte of instⁿ.

- The destination must be within same 2K block of program memory as the first byte of instⁿ following AJMP.

e.g. AJMP Address11

- Here AJMP branches to 11-bit destination address pointed by Address Label.

③ SJMP rel-addr Byte = 2, cycle = 2

- Short jump.
- Program control branches unconditionally to indicated destination rel-address.
- The branch destination is computed by adding the signed displacement in second instⁿ byte to PC, after incrementing PC twice.
- Therefore, the range of destinations allowed is from 128 bytes preceding this instⁿ 127 byte following it.
- e.g. SJMP RELADR
- SJMP branches the program control to memory address pointed by label RELADR.

④ JZ rel-addr Byte = 2, cycle = 2

- Jump if Accumulator zero.
- * - If all the bits of accumulator are '0', JZ branches to indicated destination rel-address, otherwise, it proceeds with next instⁿ.
- The branch destination is computed by adding signed relative displacement in second instⁿ byte to the PC, after incrementing PC twice.
- The Accumulator is not modified.
- No Flags are affected.
- e.g. JZ L1
- Here if contents of accumulator is '0', then JZ branches control to memory address pointed by label L1.

⑤ JNZ rel-addr Byte = 2, cycle = 2

- Jump if Accumulator not zero.
- If any bit of Accumulator is '1', JNZ branches to indicated destination address; otherwise it proceeds with the next instⁿ.
- The branch destination is computed by adding signed relative displacement in second instⁿ byte to PC, after incrementing PC twice.

32

- The Accumulator is not modified.

- No flags are affected.

E.g. JNZ L1

- If any bit of accumulator is '1', then JNZ branches control to destination address pointed by L1 Label.

* ⑥ CJNE <dest-byte>, <src-byte>, rel-addr

- compare & Jump if not equal.

* - It compares the magnitudes of first two operands i.e. destination byte & source-byte and if their values are not equal, then it branches control to destination rel-address.

- The destination is computed by adding the signed relative displacement in last instⁿ byte to PC, after incrementing PC to start of next instⁿ.

* - The carry flag is set if the unsigned integer value of <dest-byte> is less than the unsigned integer value of <src-byte>; otherwise carry flag is cleared.

- No other operand is affected.

- The first two operands allow four addressing modes :- The accumulator may be compared with any directly addressed byte or immediate data, and any indirect RAM locⁿ or working register can be compared with an immediate data.

⑦ CJNE A, direct, rel-addr Byte=3, cycle=2

- Compares contents of A with direct address / data, if not equal then branches control to destination rel-address.

⑧ CJNE A, #data, rel-addr Byte=3, cycle=2

- Compares magnitudes of A with immediate data, if not equal then branches control to destination rel-address.

③ CJNE Rn, #data, rel-addr Byte = 3, cycle = 2

- Compares content of Rn reg. where $n=0$ to 7 with immediate data and branches control to destination rel-address if magnitude of Rn $\neq$ #data is not equal.

④ CJNE @Ri, #data, rel-addr Byte = 3, cycle = 2

- Compares magnitude of contents at memory location pointed by Ri where $i=0$ or 1 with immediate data. If not equal, then branches control to destination rel-address.

⑦ DJNZ <byte>, rel-addr

- Decrement & Jump if not zero.

* - Decrements indicated byte/byte address by 1, and branches control to indicated rel-address if resulting value is not zero.

* - An original value of 00H underflows to 0FFH.

- The branch destination address is computed by adding signed relative-displacement value in last instⁿ byte to PC, after incrementing PC to first byte of following instⁿ.

- No flag are affected.

- The locⁿ decremented may be register or directly addressed byte.

⑧ DJNZ Rn, rel-addr Byte = 2, cycle = 2

- Decrement contents in Rn reg. where $n=0$ to 7 and if content of Rn is not zero, then branches control to destination rel-address.

⑨ DJNZ direct, rel-addr Byte = 3, cycle = 2

- Decrement contents in direct address by 1, and if not zero, then branches control to destination rel-address.

③④ ⑧ NOP = NO operation Byte=1, cycle=1

- Execution continues at the following instⁿ.
- other than the PC, no registers or flags are affected.
- It is sometimes used for timing delays to waste clock cycles.

⑨ LCALL addr:16 Byte=3, cycle=2

- Long call
- LCALL calls a subroutine located at indicated 16-bit address.
- The instⁿ adds three to program counter to generate the address of next instⁿ and then pushes the 16-bit result into stack (low byte first), incrementing stack pointer by two.
- The higher and lower byte of PC are then loaded, respectively, with second and third bytes of LCALL instⁿ. Program execution continues with instⁿ at this address.
- The subroutine may begin anywhere in full 64KB program memory address space.
- NO flags are affected.
- e.g. LCALL SUBR
- Here LCALL calls subroutine at memory location pointed by SUBR label & pushes current content of PC in stack memory & increment SP by two.

⑩ ACALL Addr:11 Byte=2, cycle=2

- Absolute call
- This instⁿ unconditionally calls a subroutine located at indicated address.
- The instⁿ increments the PC twice to obtain the address of following instⁿ; then pushes 16-bit result onto stack (lower byte first) and increments stack pointer twice.
- The destination address is obtained by

successively concatenating the five higher order bits of incremented PC, opcode bits 7 through 5, and second byte of instⁿ.

* - The subroutine called must start within the same 2K blocks of program memory as first byte of instⁿ following ACALL.

- NO Flags are affected.

e.g. ACALL SUBRTN

- Here ACALL calls ~~and~~ subroutine pointed by SUBRTN, this instⁿ firstly transfers the current PC contents to stack memory & increment SP by 2.

⑪ RET - Return from Subroutine Byte=1, Cycle=2.

- RET pops the higher & lower bytes of PC successively from stack, decrementing SP by two.

- Program execution continues the resulting address, generally instⁿ immediately following ACALL or LCALL.

- No flags are affected.

⑫ RETI - Return from Interrupt Byte=1, Cycle=2

- RETI pops the higher & lower bytes of PC successively from stack and restores the interrupt logic to accept additional interrupts at the same priority levels as the one just processed.

- The stack pointer is left decremented by two.

- No other registers are affected.

- The PSW is not automatically restored to its pre-interrupt status.

- Program execution continues at resulting address, which is generally instⁿ immediately after the point at which interrupt request was detected.

- If a lower or same level interrupt was pending when RETI instⁿ is executed,

(36) That one instⁿ is executed before pending interrupt is processed.

(13) JMP @A+DPTR

Byte = 1, Cycle = 2

- Jump indirect.
- This instⁿ adds 8-bit unsigned contents of Accumulator with 16-bit data pointer and loads the resulting sum to program counter.
- This is the address for sub-sequent instⁿ fetches.
- 16-bit addition is performed (modulo 2^{16}): a carry-out from low-order 8-bits propagated through higher-order bits.
- Neither accumulator nor data pointer is altered.
- NO Flags are affected.

* Write ALP to add two 8-bit number.



```
ORG 0000H ; Starting address 0000H
MOV R0, #40H; Initialize R0 at 40H address.
MOV A, @R0 ; copies content of locn of R0 to A.
INC R0 ; Increment memory locn of R0 by 1.
ADD A, @R0 ; Add content of locn of R0 with A.
INC R0 ; Increment memory locn of R0 by 1.
MOV @R0, A ; Moves Result into locn of R0.
END
```

* Write ALP to subtract 8-bit number.



```
ORG 0000H
MOV R0, #40H
MOV A, @R0
INC R0
SUBB A, @R0
INC R0
MOV @R0, A
END
```

- * Write ALP to perform multiplication of two 8-bit numbers stored at Locⁿ 15H & 16H & store result to Locⁿ 17H & 18H (use internal RAM)

```

⇒      ORG 0
        MOV R0, #15H
        MOV A, @R0
        INC R0
        MOV B, @R0
        MUL AB
        INC R0
        MOV @R0, B
        INC R0
        MOV @R0, A
        END

```

- * Write ALP to perform division of 8-bit numbers stored in internal memory of 20H & 21H & store remainder at 23H & quotient at 22H internal memory location.

```

⇒      ORG 0
        MOV R1, #20H
        MOV A, @R1
        INC R1
        MOV B, @R1
        DIV AB
        ADD INC R1
        MOV @R1, A
        INC R1
        MOV @R1, B
        END

```


* Write ALP to Find Largest Number in group of 10 Numbers stored at 30H Locⁿ & display Result at 40H locⁿ of internal memory.

```

⇒      ORG 0
        MOV R2, #10
        MOV R0, #30H
        DEC R2
        MOV A, @R0
L1: INC R0
        MOV 03, @R0
        CJNE A, 03, L2
        SJMP L3
L2: JNC L3
        MOV A, R3
L3: DJNZ R2, L1
        MOV 40H, A
        END

```

* Write ALP to Find smallest no. in group of 10 nos.

```

⇒      ORG 0
        MOV R3, #10
        MOV R1, #20H
        DEC R3
        MOV A, @R1
L1: INC R1
        MOV 04, @R1
        CJNE A, 04, L2
        SJMP L3
L2: JC L3
        MOV A, R4
L3: DJNZ R3, L1
        MOV 30H, A
        END

```

* Write ALP to arrange 5 nos. in ascending order.

```

=>      ORG 0
        MOV R3, #04
        L4: MOV R4, 03
        MOV RO, #10H
        L3: MOV A, @RO
        INC RO
        MOV B, @RO
        CJNE A, B, L1
        SJMP L2
        L1: JC L2
        MOV @RO, A
        DEC RO
        MOV @RO, B
        INC RO
        L2: DJNZ R4, L3
        DJNZ R3, L4
        END
    
```

* Write ALP to arrange 5 nos. in descending order.
Assume suitable internal memory locations.

```

=>      ORG 0
        MOV R3, #04
        L4: MOV R4, 03
        MOV RO, #10H
        L3: MOV A, @RO
        INC RO
        MOV B, @RO
        CJNE A, B, L1
        SJMP L2
        L1: JNC L2
        MOV @RO, A
        DEC RO
        MOV @RO, B
        INC RO
        L2: DJNZ R4, L3
        DJNZ R3, L4
        END
    
```

L2: DJNZ R4, L3

DJNZ R3, L4

END

* Write ALP to transfer block of 10-bytes from external RAM to internal RAM.

→

ORG 0

MOV DPTR, #0100H

MOV R1, #20H

MOV R2, #10

L1: MOVX A, @DPTR

MOV @R1, A

INC DPTR

INC R1

DJNZ R2, L1

END

* Write ALP to find even nos. in group of 10 nos from internal locⁿ 20H to 29H & show count of even no. at 40H locⁿ.

→

ORG 0

MOV R0, #20H

MOV R1, #30H

MOV R2, #10

MOV R3, #00

L1: MOV A, @R0

CLR C

RRC A

JC L2

RLC A

MOV @R1, A

INC R1

INC ~~R0~~ R3

L2: INC R0

DJNZ R2, L1

MOV 40H, R3

END

* Write ALP to perform find odd nos. in group of 10 nos. & display count.

```

=> ORG 0
    MOV R0, #20H
    MOV R1, #30H
    MOV R2, #0AH
    MOV R3, #0
L1: MOV A, @R0
    CLR C
    RRC A
    JNC L2
    RRL A
    MOV @R1, A
    INC R1
    INC R3
L2: INC R0
    DJNZ R2, L1
    MOV 40H, R3
    END.

```

* Write ALP to Find average of 10 numbers.

=> ORG 0	RET
MOV R0, #10H	L2:
MOV R2, #10	END
MOV R3, #0	
MOV B, R2	
ACALL AVG	
MOV 20H, A	
STMP L2	
AVG: MOV A, @R0	
ADDC A, R3	
INC R0	
MOV R3, A	
DJNZ R2, AVG	
DIV AB	